

Task 1

Task 1-1

```
module Adder_LookAhead8_1_1 (
    input          [ 7 : 0] a, b,
    input          [ 0 : 0] ci,    // 来自低位的进位
    output         [ 7 : 0] s,     // 和
    output         [ 0 : 0] co     // 向高位的进位
);

wire    [7:0] C;
wire    [7:0] G;
wire    [7:0] P;

assign  G = a & b;
assign  P = a ^ b;

assign  C[0] = G[0] | ( P[0] & ci );
assign  C[1] = G[1] | ( P[1] & G[0] ) | ( P[1] & P[0] & ci );
assign  C[2] = G[2] | ( P[2] & G[1] ) | ( P[2] & P[1] & G[0] ) | ( P[2] & P[1] &
P[0] & ci );
assign  C[3] = G[3] | ( P[3] & G[2] ) | ( P[3] & P[2] & G[1] ) | ( P[3] & P[2] &
P[1] & G[0] ) | ( P[3] & P[2] & P[1] & P[0] & ci );
assign  C[4] = G[4] | ( P[4] & G[3] ) | ( P[4] & P[3] & G[2] ) | ( P[4] & P[3] &
P[2] & G[1] ) | ( P[4] & P[3] & P[2] & P[1] & G[0] ) | ( P[4] & P[3] & P[2] & P[1]
& P[0] & ci );
assign  C[5] = G[5] | ( P[5] & G[4] ) | ( P[5] & P[4] & G[3] ) | ( P[5] & P[4] &
P[3] & G[2] ) | ( P[5] & P[4] & P[3] & P[2] & G[1] ) | ( P[5] & P[4] & P[3] & P[2]
& P[1] & G[0] ) | ( P[5] & P[4] & P[3] & P[2] & P[1] & P[0] & ci );
assign  C[6] = G[6] | ( P[6] & G[5] ) | ( P[6] & P[5] & G[4] ) | ( P[6] & P[5] &
P[4] & G[3] ) | ( P[6] & P[5] & P[4] & P[3] & G[2] ) | ( P[6] & P[5] & P[4] & P[3]
& P[2] & G[1] ) | ( P[6] & P[5] & P[4] & P[3] & P[2] & P[1] & G[0] ) | ( P[6] &
P[5] & P[4] & P[3] & P[2] & P[1] & P[0] & ci );
assign  C[7] = G[7] | ( P[7] & G[6] ) | ( P[7] & P[6] & G[5] ) | ( P[7] & P[6] &
P[5] & G[4] ) | ( P[7] & P[6] & P[5] & P[4] & G[3] ) | ( P[7] & P[6] & P[5] & P[4]
& P[3] & G[2] ) | ( P[7] & P[6] & P[5] & P[4] & P[3] & P[2] & G[1] ) | ( P[7] &
P[6] & P[5] & P[4] & P[3] & P[2] & P[1] & G[0] ) | ( P[7] & P[6] & P[5] & P[4] &
P[3] & P[2] & P[1] & P[0] & ci );

assign  s[0] = P[0] ^ ci;
assign  s[1] = P[1] ^ C[0];
assign  s[2] = P[2] ^ C[1];
assign  s[3] = P[3] ^ C[2];
assign  s[4] = P[4] ^ C[3];
assign  s[5] = P[5] ^ C[4];
assign  s[6] = P[6] ^ C[5];
assign  s[7] = P[7] ^ C[6];
assign  co   = C[7];

endmodule
```

```

module Adder32_1_1 (
    input                [31 : 0]    a, b,
    input                [ 0 : 0]    ci,
    output               [31 : 0]    s,
    output               [ 0 : 0]    co
);
wire    [2:0] cmid;
Adder_LookAhead8_1_1 adder8_0(
    .a(a[7:0]),
    .b(b[7:0]),
    .ci(ci),
    .s(s[7:0]),
    .co(cmid[0])
);
Adder_LookAhead8_1_1 adder8_1(
    .a(a[15:8]),
    .b(b[15:8]),
    .ci(cmid[0]),
    .s(s[15:8]),
    .co(cmid[1])
);
Adder_LookAhead8_1_1 adder8_2(
    .a(a[23:16]),
    .b(b[23:16]),
    .ci(cmid[1]),
    .s(s[23:16]),
    .co(cmid[2])
);
Adder_LookAhead8_1_1 adder8_3(
    .a(a[31:24]),
    .b(b[31:24]),
    .ci(cmid[2]),
    .s(s[31:24]),
    .co(co)
);

endmodule

module AddSub_1_1 (
    input                [31 : 0]    a, b,
    output               [31 : 0]    out,
    output               [ 0 : 0]    co
);
Adder32_1_1 addsub(
    .a(a),
    .b(~b),
    .ci(1'b1),
    .s(out),
    .co(co)
);

endmodule

```

```

module Comp_1_1 (
    input                [31 : 0]      a, b,
    output                [ 0 : 0]      ul, // 无符号数比较
    output                [ 0 : 0]      sl  // 有符号数比较
);

wire [31 : 0]  diff;
wire          sub_co;
AddSub_1_1 comp_sub(
    .a(a),
    .b(b),
    .out(diff),
    .co(sub_co)
);

assign sl = (a[31] ^ b[31]) ? (a[31] ? 1 : 0) : (diff[31] ? 1 : 0);
assign ul = ~sub_co;

endmodule

// 选择器
module Src1_1_1 (
    input                [31 : 0]      a, b,
    output                [31 : 0]      out
);

assign out = a;
endmodule

module ALU_1_1(
    input                [31 : 0]      src0,
    input                [31 : 0]      src1,
    input                [ 3 : 0]      op,
    output                [31 : 0]      res
);

wire          [12:0] sel = 13'b1 << op;

wire signed [31:0] signed_src0 = src0;

wire          [31:0] add_out;
wire          [31:0] sub_out;
wire          [ 0 :0] slt_out;
wire          [ 0 :0] sltu_out;
wire          [31:0] src1_out;

Adder32_1_1 alu_add(
    .a(src0),
    .b(src1),
    .ci(1'b0),
    .s(add_out),
    .co()
);

AddSub_1_1 alu_sub(

```

```

        .a(src0),
        .b(src1),
        .out(sub_out),
        .co()
    );

    Comp_1_1 alu_comp(
        .a(src0),
        .b(src1),
        .ul(sltu_out),
        .sl(slt_out)
    );

    Src1_1_1 alu_src1(
        .a(src0),
        .b(src1),
        .out(src1_out)
    );

    wire [31:0] and_out = src0 & src1;
    wire [31:0] or_out  = src0 | src1;
    wire [31:0] nor_out = ~(src0 | src1);
    wire [31:0] xor_out = src0 ^ src1;
    wire [31:0] sll_out = src0 << src1[4:0];
    wire [31:0] srl_out = src0 >> src1[4:0];
    wire [31:0] sra_out = signed_src0 >>> src1[4:0];

    assign res = ({32{sel[ 0]}} & add_out          ) |
                  ({32{sel[ 1]}} & sub_out          ) |
                  ({32{sel[ 2]}} & {31'b0, slt_out} ) |
                  ({32{sel[ 3]}} & {31'b0, sltu_out}) |
                  ({32{sel[ 4]}} & and_out           ) |
                  ({32{sel[ 5]}} & or_out            ) |
                  ({32{sel[ 6]}} & nor_out           ) |
                  ({32{sel[ 7]}} & xor_out           ) |
                  ({32{sel[ 8]}} & sll_out           ) |
                  ({32{sel[ 9]}} & srl_out           ) |
                  ({32{sel[10]}} & sra_out           ) |
                  ({32{sel[11]}} & src1_out          ) |

    endmodule

```

减法器的实现：根据 $-b = \sim b + 1$ 的补码转换规则，可得 $a - b = a + \sim b + 1$ ，从而将减法转化为加法；同时注意可以利用低位进位输入 ci 置 1。

比较器的实现：

- 对于有符号小于比较，如果 a , b 的符号位不同，则符号位为 1 的那个数更小；若符号位相同，则根据 $diff = a - b$ 的符号位进行判断，如果 $diff$ 的符号位为 1，则 a 更小。
- 对于无符号小于比较，直接根据 $a - b$ 的结果进位 co 进行判断，如果 co 为 1，说明 $a \geq b$ ；否则则有 $a < b$ 。

- 具体原理： $-b$ 在本问题中等同于 $2^{32} - b$ ， $a - b$ 等同于 $a + 2^{32} - b$ （33 位下的结果）。如果 $a \geq b$ ，说明第 33 位保持为 1（ 2^{32} 不会被拆散）；否则第 33 位为 0（ $a - b$ 不够减，向 2^{32} 位借位， 2^{32} 被拆散）。
- 所以 co 在这里实际上表示了借位情况，如果 $a \geq b$ ，则 $co = 1$ ；否则 $co = 0$ 。
- 另一个需要注意的点：比较器的输出只有 1 位，需要将其先扩展到 32 位再赋值给 res 。

循环右移的实现：

- 把结果分两部分考虑：一部分是右移到低位但没有循环到最高位的部分（A）；另一部分是右移出最低位并回到最高位的部分（B）。
- 对于 A，只需要执行 $a \gg b[4:0]$ 即为其答案（由于循环右移 32 位之后 a 不变，因此 $b[31:5]$ 的取值不会对结果产生任何影响）；
- 对于 B，它出现在最高位等价于在原数中将 B 左移 $32 - b[4:0]$ 位的结果。
- 最后将 A, B 按位取或即为最终的循环右移结果。

res 赋值时根据独热码 sel 每一位上的取值来选择最终的结果。

Task 1-2

```
module RF (
    input      [0 : 0]      clk      ,      // 时钟
    input      [4 : 0]      ra0, ra1,      // 读地址
    output reg [31: 0]      rd0, rd1,      // 读数据
    input      [4 : 0]      wa      ,      // 写地址
    input      [31: 0]      wd      ,      // 写数据
    input      [0 : 0]      we      // 写使能
);
reg [31:0] r[0:31];      // 寄存器堆

// 初始化所有寄存器为 0
integer i;
initial begin
    for (i = 0; i < 32; i = i + 1) begin
        r[i] = 0;
    end
end

// 读寄存器；写优先，异步
always @(*) begin
    rd0 = r[ra0];

    rd1 = r[ra1];
end

// 写寄存器（同步）
always @(posedge clk)
    if (we && wa != 0) // 只有写使能有效且地址非 0 时才写入
        r[wa] <= wd;
endmodule
```

寄存器实现异步读的思路：在读操作时，直接使用组合逻辑赋值 `rd0 = r[ra0]` 和 `rd1 = r[ra1]`。这种方式下，读操作不受时钟信号边沿的控制，地址信号发生变化后，读取的内容能够立刻在同一周期内反映在输出端口上。与此同时，寄存器堆的写操作同步执行，保证了寄存器状态更新的稳定性。

值得注意的是：代码中的读出端口 `rd0` 和 `rd1` 必须被声明为 `reg` 类型并置于 `always @(*)` 块中赋值，而不能被声明为 `wire` 类型并使用 `assign` 进行连续赋值。如果使用 `wire` 类型直接赋连续值，在同一个时钟上升沿对同一地址进行同时读写（读写冲突）时，内部数组的同步更新与输出的连续读取会在事件触发机制下**产生竞争-冒险**，导致读取端口捕捉到电平更替的过渡态，从而在仿真中出现 `xx`（未知值）的不稳定状态；而将其声明为 `reg` 并置放在 `always @(*)` 块内，则能够依靠仿真器的过程赋值机制有效隔离这种竞争冲突，避免不稳定的 `xx` 状态的出现。

Task 2

本任务使用的 testbench 文件如下：

```
`timescale 1ns / 1ps

module memory_compare_tb();
    reg clk;
    reg we;
    reg [9:0] addr;
    reg [31:0] din;
    wire [31:0] dout_dist;          // Distributed RAM 的输出
    wire [31:0] dout_block_wfirst; // 写优先 BRAM 的输出
    wire [31:0] dout_block_rfirst; // 读优先 BRAM 的输出

    // 实例化 Distributed RAM
    dist_mem_gen_0 dist_mem (
        .a(addr), .d(din), .clk(clk), .we(we), .spo(dout_dist)
    );

    // 实例化 Block RAM (写优先)
    blk_mem_gen_0 blk_mem_wfirst (
        .clka(clk), .wea(we), .addra(addr), .dina(din), .douta(dout_block_wfirst)
    );

    // 实例化 Block RAM (读优先)
    blk_mem_gen_1 blk_mem_rfirst (
        .clka(clk), .wea(we), .addra(addr), .dina(din), .douta(dout_block_rfirst)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0; we = 0; addr = 0; din = 0;
        #20;

        // 向目标地址 10 写入数据 0xABCD1234 (同步)
        @(posedge clk); // 等待时钟上升沿 (对齐)
        addr = 10'd10; din = 32'hABCD1234; we = 1;
        @(posedge clk); // 该上升沿 Distributed RAM 写入新值并立即读出新值
    end
endmodule
```

```

    @(posedge clk); // 该上升沿 BRAM 写入新值，写优先 BRAM 读出新值
    @(posedge clk); // 该上升沿读优先 BRAM 读出新值
    #5;
    we = 0;

    #10;
    // 在目标地址不变的情况下，写入新数据
    @(posedge clk);
    din = 32'h1234ABCD; we = 1;
    @(posedge clk); // 该上升沿 Distributed RAM 写入新值并立即读出新值
    @(posedge clk); // 该上升沿 BRAM 写入新值，写优先 BRAM 读出新值
    @(posedge clk); // 该上升沿读优先 BRAM 读出新值
    #5;
    we = 0;

    // 改变目标地址到 0
    @(posedge clk);
    #5;
    addr = 10'd0; // distributed RAM 立即读出新值
    @(posedge clk); // 该上升沿两种 BRAM 仍未读出新值
    @(posedge clk); // 该上升沿两种 BRAM 同时读出新值

    #20 $finish;
end
endmodule

```

在仿真波形中看到，在一个时钟上升沿尝试写入的同时读数据时，Distributed RAM 在下一个上升沿读出新值，写优先的 BRAM 在下两个上升沿读出新值，读优先的 BRAM 在下三个上升沿读出新值；当只做读取数据的操作时，Distributed RAM 立即读出新值，BRAM 在下两个上升沿读出新值。

- 当 BRAM 端口设置中的 **Primitives Output Register** 开关被关闭时，两种 BRAM 的时序均**提前一个时钟周期**，即：同时读写时写优先的 BRAM 在下一个上升沿读出新值，读优先的 BRAM 在下两个上升沿读出新值；仅读取时，BRAM 在下一个上升沿读出新值。

Distributed RAM 的读取基于纯粹的组合逻辑，因此可以即时响应任何时候的仅读取请求。对于同时读写的请求，由于 Distributed RAM 采用异步读取、同步写入，因此在当前上升沿新数据即被写入，并通过一个时钟周期的稳定过程后被读出。

打开 **Primitives Output Register** 开关时，BRAM 为了保证在高工作频率下仍能满足建立/保持时间要求，在时序上表现出多周期的潜伏期（具体来说，在本实验的仿真中体现为 2 个时钟周期）。在第一个时钟上升沿，BRAM 主要完成**完成输入信号的采样与同步**，当系统时钟的上升沿到来时，块式存储器的输入端寄存器会将外部总线上的地址、写使能信号以及待写入的数据同步捕获到内部逻辑中。第二个时钟上升沿 BRAM 仍未读出，与 BRAM 为了隔离关键路径而设置数据输出流水线有关。内部核心阵列在第一个时钟周期内完成数据访问后，该数据会被送至输出端，并在第二个时钟上升沿被**输出寄存器**稳定锁存，随后才真正呈现在外部数据总线上。因此，在仅读取请求中，两种 BRAM 都在第二个上升沿处读出新值，而在同时读写请求中，写优先 BRAM 在第二个上升沿处读出新值，读优先 BRAM 在第三个上升沿处读出新值（还需要经过一个时钟周期才能读取新写入该地址的值）。

Task 3

Task 3-1

冒泡排序的核心代码如下：

```
// -----  
// 主控时序逻辑  
// -----  
always @(posedge clk or negedge rstn) begin  
    if (!rstn) begin  
        current_state    <= S_IDLE;  
        i                <= 0;  
        j                <= 1023;  
        count            <= 0;  
        bram_wea         <= 0;  
        bram_web         <= 0;  
        bram_addra       <= 0;  
        bram_addrb       <= 0;  
        bram_dina        <= 0;  
        bram_dinb        <= 0;  
        temp             <= 0;  
    end  
    else begin  
        current_state <= next_state;  
        case (current_state)  
            S_IDLE: begin  
                bram_wea <= 0;  
                bram_web <= 0;  
  
                // 在空闲状态，将端口 A 开放给拨码开关以便查看数据  
                bram_addra <= addr;  
  
                if (start) begin // 检测到开始排序信号，把所有排序中使用的辅助  
                    变量初始化  
                        i    <= 0;  
                        j    <= 1023;  
                        count <= 0;  
                    end  
                end  
  
            S_READ: begin  
                count <= count + 1;  
                bram_wea <= 0;  
                bram_web <= 0;  
                // 发送相邻两个数据的地址给 BRAM 端口 A 和 B  
                bram_addra <= j - 1;  
                bram_addrb <= j;  
            end  
  
            S_RWAIT: begin // 缓冲 1 个时钟周期  
                count <= count + 1;  
            end  
  
            S_CMP: begin // 使用锁存值比较
```

```

count <= count + 1;
if ((mode == 1'b1 && bram_douta > bram_doutb) ||
    (mode == 1'b0 && bram_douta < bram_doutb)) begin
    // 需要交换：先将 A 端口读出的数据暂存
    temp <= bram_douta;

    // 先向 A 端口写入
    bram_addra <= j - 1;
    bram_dina <= bram_doutb;
    bram_wea <= 1;
    bram_web <= 0;
end
else begin
    // 不需要交换，直接跳转到下一个元素
    if (j > i + 1) begin
        j <= j - 1;
    end
    else begin
        if (i < 1022) begin
            i <= i + 1;
            j <= 1023;
        end
    end
end
end
end

S_WRITE1: begin
    count <= count + 1;
    // 写只需一个周期，立刻关闭写使能
    bram_wea <= 0;

    // 再向 B 端口写入
    bram_addrb <= j;
    bram_dinb <= temp;
    bram_web <= 1;
end

S_WRITE2: begin
    count <= count + 1;
    // 写只需一个周期，立刻关闭写使能
    bram_web <= 0;
end

S_WWAIT: begin
    count <= count + 1;

    // 索引跳转逻辑
    if (j > i + 1) begin
        j <= j - 1;
    end
    else begin
        if (i < 1022) begin
            i <= i + 1;
            j <= 1023;
        end
    end
end

```

```

        end
    end
end

S_DONE: begin
    // 排序完成后，再次将端口 A 开放给拨码开关以查看数据
    bram_addra <= addr;
end
endcase
end
end

// -----
// 状态转换组合逻辑
// -----
always @(*) begin
    next_state = current_state;
    case (current_state)
        S_IDLE:
            if (start)
                next_state = S_READ;
            else
                next_state = S_IDLE;
        S_READ: // 从内存中读取，跳转到等待周期
            next_state = S_RWAIT;
        S_RWAIT: // 等待 1 个时钟周期，使输出端口数据稳定
            next_state = S_CMP;
        S_CMP:
            if ((mode == 1'b1 && bram_douta > bram_doutb) || // 升序情况下
(1), 前面的数比后面的数更大
                (mode == 1'b0 && bram_douta < bram_doutb)) // 降序情况下
(0), 前面的数比后面的数更小
                next_state = S_WRITE1; // 需要进行数
据交换，先准备写入
            else begin // 否则，直接进行下一轮比较
                if (j > i + 1 || i < 1022) // 完整一轮的比较仍未结束，或者后面
还有新一轮比较需要进行
                    next_state = S_READ;
                else // 比较完成
                    next_state = S_DONE;
            end
        S_WRITE1:
            next_state = S_WRITE2;
        S_WRITE2:
            next_state = S_WWAIT;
        S_WWAIT:
            if (j > i + 1 || i < 1022) // 完整一轮的比较仍未结束，或者后面还有
新一轮比较需要进行
                next_state = S_READ;
            else // 比较完成
                next_state = S_DONE;
        S_DONE: begin
            next_state = S_DONE; // 保持在 DONE 状态，使板上 done 信号对
应 LED 持续点亮
    end
end
end

```

```

        end
        default: // 对于其它非法状态，直接跳转到 IDLE
            next_state = S_IDLE;
        endcase
    end

    // -----
    // 输出逻辑
    // -----
    always @(*) begin
        done = (current_state == S_DONE);
    end

endmodule

```

在本任务的实现中，使用 IP 核例化的写优先真双端口读写 BRAM。

排序模块中，设计的状态机包括 IDLE、READ、RWAIT、CMP、WRITE1、WRITE2、WWAIT、DONE 等状态。复位后，状态机默认处于 IDLE 状态。

在排序过程中使用了两个临时变量用于指示当前的排序状态。外层循环 i 从 0 递增到 1022 (i 以前的数据已经有序)，内层循环 j 从 1023 递减到 $i + 1$ (把当前最大 / 小的数据移动到有序区段末尾)，每次比较相邻两个元素并根据排序模式决定是否交换。

状态机的具体转换流程如下：

1. 在 IDLE 状态，状态机将 BRAM 端口 A 开放给外部拨码开关，以使用户查看任意地址上的数据。当检测到 `start` 信号有效时，状态机初始化循环变量 $i = 0$ 和 $j = 1023$ ，并跳转到 READ 状态开始排序。
2. 在 READ 状态，状态机通过 BRAM 的两个端口分别发送地址 $j - 1$ 和 j ，准备读取相邻的两个数据元素。地址发送完成后，立即跳转到 RWAIT 状态。
3. BRAM 采用同步读取方式，需要一个时钟周期的延迟才能将数据稳定输出到 `douta` 和 `doutb` 端口。因此，状态机在 RWAIT 状态等待一个时钟周期，等待输出端口的数据稳定后，再跳转到 CMP 状态进行数据比较。
4. 在 CMP 状态，排序模块根据排序模式 `mode` 对读取的两个数据进行比较：
 - 若 `mode = 1` (升序) 且 `douta > doutb`，或 `mode = 0` (降序) 且 `douta < doutb`，则需要交换两个数据，将 `douta` 存入 `temp`、设置端口 A 的写入地址为 $j - 1$ ，写入数据为 `douta` 后，跳转到 WRITE 状态；
 - 否则，不需要交换。若此时比较仍未结束 ($j > i + 1$ 或 $i < 1022$)，则更新索引并跳转回 READ 状态继续下一轮比较；若所有比较已完成，则跳转到 DONE 状态。
5. 在 WRITE1 状态，关闭 A 端口写入使能，同时设置端口 B 的写入地址为 j 、写入数据为 `temp`，之后跳转到 WRITE2 状态。
6. 在 WRITE2 状态，`doutb` 通过端口 A 写入地址 $j - 1$ ；关闭 B 端口写入使能。之后跳转到 WWAIT 状态。
7. 在 WWAIT 状态，`temp(=douta)` 通过端口 B 写入地址 j 。同时根据循环索引判断下一步的跳转：
 - 若比较仍未结束 ($j > i + 1$ 或 $i < 1022$)，则更新索引并跳转回 READ 状态；
 - 若所有比较已完成，则跳转到 DONE 状态。
8. 在 DONE 状态时，状态机将 `done` 信号置为有效，并将 BRAM 端口 A 再次开放给外部拨码开关以便查看排序后的数据。为了保证上板运行时 `done` 对应的信号能持续有效，DONE 状态将一直维持，直到 `rstn` 信号重置整个状态机。

Task 3-2

LUT、FF 等资源的使用情况如下图：

Utilization		Post-Synthesis Post-Implementation	
		Graph Table	
Resource	Utilization	Available	Utilization %
LUT	123	63400	0.19
FF	186	126800	0.15
BRAM	1	135	0.74
IO	46	210	21.90
BUFG	1	32	3.13

经过测试，当时钟周期为 7.123, ns 时，WNS 达到 0.076, ns，故电路正常工作的最高时钟频率约为 141.90, MHz。

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.076 ns	Worst Hold Slack (WHS): 0.099 ns	Worst Pulse Width Slack (WPWS): 3.061 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 456	Total Number of Endpoints: 456	Total Number of Endpoints: 189
All user specified timing constraints are met.		

Task X

Task X-2

代码解释

快速排序的核心代码如下：

```
// -----  
// 主控时序逻辑  
// -----  
always @(posedge clk or negedge rstn) begin  
    if (!rstn) begin  
        current_state <= S_IDLE;  
        count         <= 0;  
        sp             <= 0;  
        bram_wea       <= 0;  
        bram_web       <= 0;  
        bram_addra     <= 0;  
        bram_addrb     <= 0;  
        bram_dina       <= 0;  
        bram_dinb       <= 0;
```

```

low          <= 0;
high         <= 0;
i            <= 0;
j            <= 0;
pivot_idx    <= 0;
pivot_val    <= 0;
val_j        <= 0;
val_i        <= 0;
val_i1       <= 0;
end
else begin
    current_state <= next_state;
    case (current_state)
        S_IDLE: begin
            bram_wea    <= 0;
            bram_web    <= 0;
            bram_addra   <= addr; // 开放给拨码开关查看

            if (start) begin
                sp        <= 1;
                stack[0]  <= {11'sd0, 11'sd1023}; // 初始化压入整个数组范围

                count     <= 0;
            end
        end
        S_POP: begin
            count <= count + 1;
            bram_wea <= 0;
            bram_web <= 0;
            if (sp != 0) begin // 栈非空, 读出 {low, high} 数据对并出栈
                low  <= stack[sp-1][21:11];
                high <= stack[sp-1][10:0];
                sp   <= sp - 1;
            end
        end
        S_READ_PIVOT: begin
            count <= count + 1;
            bram_addra <= high; // 固定取最后一个元素作为基准
        end
        S_WAIT_PIVOT: begin // 等待读取 bram[high]
            count <= count + 1;
        end
        S_LATCH_PIVOT: begin
            count <= count + 1;
            pivot_val <= bram_douta;

            // 初始化 pivot 左侧下标和遍历起点
            i <= low - 1;
            j <= low;
        end
    end case
end

```

```

S_LOOP_COND: begin
    count <= count + 1;
    if (j < high) begin // 还没有遍历完所有元素
        bram_addra <= j;
    end
end

S_WAIT_J: begin // 等待读取 bram[j]
    count <= count + 1;
end

S_LATCH_J: begin
    count <= count + 1;
    val_j <= bram_douta;
end

S_CMP: begin
    count <= count + 1;
    // 判断是否满足交换条件
    if ((mode == 1'b1 && val_j <= pivot_val) ||
        (mode == 1'b0 && val_j >= pivot_val)) begin
        i <= i + 1;
        bram_addrb <= i + 1; // 准备读取 bram[i] 进行交换; 交换后
bram[i] 仍需要判断, j 不变
    end
    else begin
        j <= j + 1;
    end
end

S_WAIT_I: begin // 等待读取 bram[i]
    count <= count + 1;
end

S_LATCH_I: begin
    count <= count + 1;
    val_i <= bram_doutb;
end

S_SWAP1: begin
    count <= count + 1;
    bram_wea    <= 1;
    bram_addra  <= i;
    bram_dina   <= val_j;

    bram_web    <= 0;
    bram_addrb  <= j;
    bram_dinb   <= val_i;
end

S_SWAP2: begin
    count <= count + 1;
    bram_wea    <= 0;

```

```

        bram_web    <= 1;
        j <= j + 1;
    end

    S_SWAP_WAIT1: begin
        count <= count + 1;
        bram_web <= 0;
    end

    S_SWAP_WAIT2: begin
        count <= count + 1;
    end

    // --- 循环结束后, 将基准元素放到 i+1 索引处---
    S_SWAP_PIVOT_READ: begin
        count <= count + 1;
        pivot_idx <= i + 1;
        bram_addra <= i + 1;
    end

    S_WAIT_PIVOT_SWAP: begin // 等待读取 bram[i+1]
        count <= count + 1;
    end

    S_LATCH_PIVOT_SWAP: begin
        count <= count + 1;
        val_i1 <= bram_douta;
    end

```

S_SWAP_PIVOT_WRITE1: begin // 先写 A 端口, 把 pivot 归位; 同时可以
 先把 B 端口待写入的数据准备好

```

        count <= count + 1;
        bram_wea <= 1;
        bram_addra <= pivot_idx;
        bram_dina <= pivot_val;

        bram_web <= 0;
        bram_addrb <= high;
        bram_dinb <= val_i1;
    end

    S_SWAP_PIVOT_WRITE2: begin // 再写 B 端口
        count <= count + 1;
        bram_wea <= 0;
        bram_web <= 1;
    end

    S_SWAP_PIVOT_WAIT1: begin
        count <= count + 1;
        bram_web <= 0;
    end

    S_SWAP_PIVOT_WAIT2: begin
        count <= count + 1;
    end

```

```

end

// --- 将产生的新子数组索引压入栈 ---
S_PUSH: begin
    count <= count + 1;
    bram_wea <= 0; bram_web <= 0;

    if (low < pivot_idx - 1 && pivot_idx + 1 < high) begin // 左右
        区间同时非空
            stack[sp] <= {low, pivot_idx - 1'sd1};
            stack[sp+1] <= {pivot_idx + 1'sd1, high};
            sp <= sp + 2;
        end
        else if (low < pivot_idx - 1) begin // 仅
            左区间非空
                stack[sp] <= {low, pivot_idx - 1'sd1};
                sp <= sp + 1;
            end
            else if (pivot_idx + 1 < high) begin // 仅
                右区间非空
                    stack[sp] <= {pivot_idx + 1'sd1, high};
                    sp <= sp + 1;
                end
            end
        end

    S_DONE: begin
        bram_addra <= addr; // 开放查看
    end
endcase
end
end

// -----
// 状态转换组合逻辑
// -----
always @(*) begin
    next_state = current_state;
    case (current_state)
        S_IDLE:
            if (start)
                next_state = S_POP;
            else
                next_state = S_IDLE;

        S_POP:
            if (sp == 0) // 栈空, 则排序完成
                next_state = S_DONE;
            else
                next_state = S_READ_PIVOT;

        S_READ_PIVOT: // 读出基准元素
            next_state = S_WAIT_PIVOT;

        S_WAIT_PIVOT: // 等待读出 bram[high]

```

```

        next_state = S_LATCH_PIVOT;

S_LATCH_PIVOT:
    next_state = S_LOOP_COND;

S_LOOP_COND:
    if (j < high)    // 还没有遍历完所有元素
        next_state = S_WAIT_J;
    else            // 遍历完成，下一步将基准元素放到正确位置
        next_state = S_SWAP_PIVOT_READ;

S_WAIT_J:          // 等待读出 bram[j]
    next_state = S_LATCH_J;

S_LATCH_J:
    next_state = S_CMP;

S_CMP:
    if ((mode == 1'b1 && val_j <= pivot_val) ||           // 升序情况下
(1), 当前的数小于 pivot
        (mode == 1'b0 && val_j >= pivot_val))           // 降序情况下
(0), 当前的数大于 pivot
        next_state = S_WAIT_I;
    else
        next_state = S_LOOP_COND;

S_WAIT_I:          // 等待读出 bram[i]
    next_state = S_LATCH_I;

S_LATCH_I:
    next_state = S_SWAP1;

S_SWAP1:
    next_state = S_SWAP2;

S_SWAP2:
    next_state = S_SWAP_WAIT1;

S_SWAP_WAIT1:
    next_state = S_SWAP_WAIT2;

S_SWAP_WAIT2:
    next_state = S_LOOP_COND;

S_SWAP_PIVOT_READ:
    next_state = S_WAIT_PIVOT_SWAP;

S_WAIT_PIVOT_SWAP: // 等待读出 bram[i+1]
    next_state = S_LATCH_PIVOT_SWAP;

S_LATCH_PIVOT_SWAP:
    next_state = S_SWAP_PIVOT_WRITE1;

S_SWAP_PIVOT_WRITE1:

```

```

        next_state = S_SWAP_PIVOT_WRITE2;

S_SWAP_PIVOT_WRITE2:
    next_state = S_SWAP_PIVOT_WAIT1;

S_SWAP_PIVOT_WAIT1:
    next_state = S_SWAP_PIVOT_WAIT2;

S_SWAP_PIVOT_WAIT2:
    next_state = S_PUSH;

S_PUSH:
    next_state = S_POP;

S_DONE:
    next_state = S_IDLE;
default:
    next_state = S_IDLE;
endcase
end

// -----
// 输出逻辑
// -----
always @(*) begin
    done = (current_state == S_DONE);
end

endmodule

```

在本任务的实现中，使用自行创建的读优先双端口读写 BRAM。

排序模块基于快速排序算法，固定每轮排序的 pivot 为区间最后一个元素，通过硬件栈存储待处理的子数组索引，实现算法的非递归化。在排序模块中设计的状态机包括 24 种状态；复位后，状态机默认处于 IDLE 状态。

为实现快速排序的递归逻辑，模块内维护一个深度为 1024 的硬件栈 `stack`，每个栈元素存储一个索引对 `{low, high}`，表示待排序的子数组范围。栈顶指针 `sp` 用于追踪栈的当前深度。

状态机的具体转换流程如下：

1. 在 IDLE 状态，状态机将 BRAM 端口 A 开放给外部拨码开关，以使用户查看任意地址上的数据。当检测到 `start` 信号有效时，状态机将初始区间 `{0, 1023}` 压入栈中，并跳转到 POP 状态开始排序。
2. 在 POP 状态，状态机尝试从栈顶弹出一组索引对 `{low, high}`。若栈为空 (`sp == 0`)，说明所有子数组均已处理完毕，排序完成，跳转到 DONE 状态；否则，跳转到 READ_PIVOT 状态开始分区操作。
3. 在 READ_PIVOT 状态，状态机通过 BRAM 端口 A 发送地址 `high`，读取区间末尾元素 `bram[high]` 作为基准 (pivot)。地址发送完成后，立即跳转到 WAIT_PIVOT 状态。
4. BRAM 采用同步读取方式，需要一个时钟周期的延迟才能将数据稳定输出。因此，状态机在 WAIT_PIVOT 状态等待一个时钟周期，等待输出端口的数据稳定后，再跳转到 LATCH_PIVOT 状态。
5. 在 LATCH_PIVOT 状态，状态机将基准值锁存到 `pivot_val`，并初始化分区索引 `i = low - 1` 和扫描索引 `j = low`，之后跳转到 LOOP_COND 状态。
6. 在 LOOP_COND 状态，状态机判断分区循环条件 `j < high`：

- 若条件成立，则通过 BRAM 端口 A 发送地址 j 读取 $bram[j]$ ，并跳转到 `WAIT_J` 状态继续分区；
 - 若条件不成立且 $i + 1 == high$ （说明 `pivot` 已经在正确位置 `high`，无需归位交换），此时同时将 $pivot_idx = i + 1$ 锁存，直接跳转到 `PUSH` 状态；
 - 若条件不成立且 $i + 1 != high$ ，则跳转到 `SWAP_PIVOT_READ` 状态将基准放到正确位置。
7. 在 `WAIT_J` 状态，状态机等待 BRAM 将 $bram[j]$ 的数据稳定输出，等待一个时钟周期后跳转到 `LATCH_J` 状态。
 8. 在 `LATCH_J` 状态，状态机将 $bram[j]$ 锁存到 val_j ，跳转到 `CMP` 状态进行比较判断。
 9. 在 `CMP` 状态，状态机根据排序模式 `mode` 比较 val_j 与 $pivot_val$ ：
 - 若满足交换条件（升序时 $val_j <= pivot_val$ ，降序时 $val_j >= pivot_val$ ），则将 i 自增：
 - 若此时 $i + 1 == j$ （即与自身交换，无意义），则直接将 j 自增并跳转回 `LOOP_COND` 状态，**跳过本次交换**；
 - 否则，通过 BRAM 端口 B 发送地址 $i + 1$ 准备读取 $bram[i + 1]$ ，跳转到 `WAIT_I` 状态；
 - 否则，直接将 j 自增并跳转回 `LOOP_COND` 状态继续下一轮循环。
 10. 在 `WAIT_I` 状态，状态机等待 BRAM 将 $bram[i + 1]$ 的数据稳定输出，等待一个时钟周期后跳转到 `LATCH_I` 状态。
 11. 在 `LATCH_I` 状态，状态机将 $bram[i + 1]$ 锁存到 val_i ，跳转到 `SWAP1` 状态执行交换。
 12. 在 `SWAP1` 状态，仅通过端口 A 写入 $bram[i] = val_j$ （打开 A 写入使能，关闭 B 写入使能）；同时预填 B 端口的写入数据 val_i ，但**不预设 B 端口地址**（以防止和 A 端口产生同地址冲突）。随后跳转到 `SWAP2` 状态。
 13. 在 `SWAP2` 状态，关闭 A 端口的写入，设置 B 端口写入地址为 j 、写入数据为 val_i ，并打开 B 端口写入（此时 A 端口已关闭，两端口不会产生冲突）；同时将 j 自增，随后跳转到 `SWAP_WAIT1` 状态。
 14. 在 `SWAP_WAIT1` 状态，关闭 B 端口的写入，随后跳转到 `SWAP_WAIT2` 状态。
 15. 在 `SWAP_WAIT2` 状态，确保数据已稳定写入 BRAM。之后跳转回 `LOOP_COND`。
 16. 在 `SWAP_PIVOT_READ` 状态，分区循环已结束，状态机准备将基准放到正确位置 $i + 1$ ，同时将 $pivot_idx = i + 1$ 锁存。通过 BRAM 端口 A 发送地址 $i + 1$ 读取 $bram[i + 1]$ ，跳转到 `WAIT_PIVOT_SWAP` 状态。
 17. 在 `WAIT_PIVOT_SWAP` 状态，状态机等待 BRAM 将 $bram[i + 1]$ 的数据稳定输出，等待一个时钟周期后跳转到 `LATCH_PIVOT_SWAP` 状态。
 18. 在 `LATCH_PIVOT_SWAP` 状态，状态机将 $bram[i + 1]$ 锁存到 val_i1 ，跳转到 `SWAP_PIVOT_WRITE1` 状态执行基准交换。
 19. 在 `SWAP_PIVOT_WRITE1` 状态，仅通过端口 A 写入 $bram[pivot_idx] = pivot_val$ （打开 A 写入使能，关闭 B 写入使能）；同时预填 B 端口的写入数据 val_i1 ，但**不预设 B 端口地址**，以防止 $pivot_idx == high$ 时两端口同地址冲突。之后跳转到 `SWAP_PIVOT_WRITE2` 状态。
 20. 在 `SWAP_PIVOT_WRITE2` 状态，关闭 A 端口写入，设置 B 端口写入地址为 `high`，并打开 B 端口写入（此时 A 端口已关闭，两端口不会产生冲突），随后跳转到 `SWAP_PIVOT_WAIT1` 状态。
 21. 在 `SWAP_PIVOT_WAIT1` 状态，关闭 B 端口写入，随后跳转到 `SWAP_PIVOT_WAIT2` 状态。
 22. 在 `SWAP_PIVOT_WAIT2` 状态，确保数据已稳定写入 BRAM。之后跳转到 `PUSH`。
 23. 在 `PUSH` 状态，状态机根据分区结果，将新产生的子数组索引压入栈中：
 - 若 $low < pivot_idx - 1$ ，则左子数组 $[low, pivot_idx - 1]$ 有效，将其压栈；
 - 若 $pivot_idx + 1 < high$ ，则右子数组 $[pivot_idx + 1, high]$ 有效，将其压栈；
 - 压栈完成后，跳转回 `POP` 状态处理下一个子数组。
 24. 在 `DONE` 状态时，状态机将 `done` 信号置为有效，并将 BRAM 端口 A 再次开放给外部拨码开关以便查看排序后的数据。为保证上板运行时 `done` 对应信号持续有效，`DONE` 状态将一直维持，直到 `rstn` 信号重置整个状态机。

资源使用、性能比较

LUT、FF 等资源使用情况

快速排序实现的 LUT、FF 等资源使用情况如下：

Utilization

Post-Synthesis | Post-Implementation

Graph | Table

Resource	Utilization	Available	Utilization %
LUT	33212	63400	52.38
FF	23407	126800	18.46
BRAM	1	135	0.74
IO	46	210	21.90
BUFG	1	32	3.13

最高时钟频率

经过测试，当时钟周期为 5.9, ns 时，WNS 达到 0.076, ns，故电路正常工作的最高时钟频率约为 171.70, MHz。

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.076 ns	Worst Hold Slack (WHS): 0.051 ns	Worst Pulse Width Slack (WPWS): 2.450 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 944	Total Number of Endpoints: 944	Total Number of Endpoints: 401
All user specified timing constraints are met.		

排序速度

对 data.coe 中的数据进行降序冒泡排序时，共花费 0x243327=2372391 个时钟周期；进行升序冒泡排序时，共花费 0x23BAD9=2341593 个时钟周期。

对 data.coe 中的数据进行降序快速排序时，共花费 0x14C64=85092 个时钟周期；进行升序快速排序时，共花费 0x12B6C=76652 个时钟周期。

从硬件实现角度分析，快速排序相比冒泡排序需要更多的电路资源，主要体现在以下几个方面：

- 1. **状态机复杂度**：冒泡排序的状态机仅有 7 个状态，使用 3 位状态寄存器即可表示；而快速排序的状态机包含 24 个状态，需要 5 位状态寄存器。更复杂的状态转换逻辑会消耗更多的组合逻辑资源。
- 2. **存储资源**：非递归化的快速排序过程需要额外的硬件栈来存储待处理的子数组索引。在实现本任务时使用的栈深度为 1024，每个栈元素存储一个 22 位的索引对 {low, high}，总共需要 1024 × 22 = 22528 位的存储空间。此外，快速排序还需要维护多个辅助寄存器（pivot_val、val_j、val_i、val_i1 等）用于暂存中间数据。

3. **运算逻辑**：快速排序在分区过程中需要进行复杂的比较和索引计算，涉及有符号数的计算；冒泡排序的运算逻辑相对更简单，主要涉及无符号数的递减和比较操作。

从综合实现结果来看，冒泡排序的最高工作频率约为 141.90, MHz，而快速排序的最高工作频率约为 171.70, MHz。尽管快速排序包含更复杂的状态转换逻辑、硬件栈的读写以及有符号数运算电路，其关键路径反而更短，主要原因可能包括：

- 快速排序在单个状态内执行的操作被拆分得更细致，减少了单周期内组合逻辑的级联延迟；
- BRAM 读写、比较等操作的流水线排布更加合理，关键路径得到了综合器的更好优化。

从算法复杂度角度分析，快速排序的平均时间复杂度为 $O(n \log n)$ ，而冒泡排序的时间复杂度为 $O(n^2)$ 。对于大规模数据，快速排序所需时钟周期数（万周期级）仅为冒泡排序（百万周期级）的极小部分。结合其更高的最高工作频率，快速排序在整体耗时上具有绝对优势。

总的来看，两种排序算法的硬件实现分别呈现了不同的开销与性能特征。冒泡排序实现简单、状态机规模小、LUT 和 FF 等逻辑资源占用少，适合**硬件资源受限**且对排序速度要求不高的场景；快速排序虽然需要较多的额外硬件资源（如维护硬件栈的存储资源、更多状态寄存器），但其凭借极高的算法效率和更好的时序表现，在**大规模数据排序**中能够充分弥补硬件开销，适合对排序性能有较高要求的应用场景。